

SOFTWARE ENGINEERING

04

Introduction to OOP Programming

Stop writing code that fights itself — learn to model the real world with objects, classes, and clean design.

DURATION	LEVEL	MODULE	UPDATED
8 Hours · 7 Sessions + 1 Mini Project	Intermediate	Module 4	April 2026

IN THIS MODULE

01	Procedural vs Object Thinking
02	Classes and Objects
03	Constructors & Destructors
04	Inheritance
05	Polymorphism
06	Encapsulation & Abstraction
07	File Handling in C++
MP	Mini Project 2 — Creator Dashboard Lite

SESSION

01

60 MIN · FUNDAMENTALS

Procedural vs Object Thinking

◆ Learning Objectives

- Identify the core limitations of procedural programming in large programs
- Explain the four pillars of Object-Oriented Programming
- Compare how a Task List problem is solved procedurally versus with objects
- Recognise real-world entities as classes and objects
- Describe why C++ was designed as an extension of C

01 The Problem with Procedural Code

In C, data and functions live separately. As a program grows, any function can accidentally modify any variable — there is no wall protecting your data. Imagine building a student management system for a Gujarat college: 500 students, dozens of functions, and nothing stopping a `calculate_marks()` function from quietly corrupting a student's name.

CONCEPT — Procedural Programming

Procedural programming executes code as a top-down sequence of steps (functions) that operate on shared data. It works well for small programs but becomes fragile as complexity grows — there is no built-in way to group data with the functions that manage it.

Procedural (C style)

Data and functions are separate. Any function can access any variable. Adding a new entity type (e.g. 'Teacher') requires rewriting many existing functions. Difficult to scale safely in a team.

Object-Oriented (C++ style)

Data and related functions are bundled inside a class. Access is controlled by the programmer. New entity types are added as new classes that inherit from existing ones — no rewriting required.

02 Thinking in Objects

Object-Oriented Programming asks you to model the world as it really is. Every entity — a student, an order, a delivery partner — becomes an object that carries both its data (attributes) and its behaviour (methods) together. Think of Zomato: a `DeliveryPartner` object knows its name and GPS coordinates, and can call `update_location()` — all bundled in one self-contained unit.

CONCEPT — The Four Pillars of OOP

1. Encapsulation — bundle data and functions together; control who can access what.

2. Inheritance — new classes reuse and extend existing ones without rewriting code.
3. Polymorphism — one interface, many behaviours depending on the type of object.
4. Abstraction — expose only what is necessary, hide the internal complexity.

03 C vs C++: Task List Side-by-Side

The clearest way to feel the difference is to write the same small program twice. Below is a Task List — storing a task name and a completion flag — written both in C procedural style and C++ OOP style.

```
// ---- C (procedural style) ----
char taskName[50] = "Study OOP";
int isDone = 0;
void markDone() { isDone = 1; }
```

Global data — any function can overwrite this
No link to taskName — separate variable
Function lives completely apart from data

```
// ---- C++ (OOP style) ----
class Task {
public:
    string name;
    bool isDone = false;
    void markDone() { isDone = true; }
};
```

Class groups data + behaviour together
Access specifier — members visible outside the class
Attribute: the task name
Attribute: completion flag
Method lives inside the class
Semicolon is mandatory after class body

TRY THIS LIVE

Type the C++ Task class above in VS Code. In main(), create two Task objects with different names. Call markDone() on only one. Print both isDone values. Notice that each object has its own independent copy of the data — changing one does not affect the other.

04 Why C++?

C++ is C with OOP layered on top — you can still write procedural code inside a .cpp file. This makes the transition comfortable: you keep all your C knowledge from Module 3 and layer OOP on top one concept at a time. System software, game engines, and real-time trading platforms still rely on C++, making it a highly valued language at Indian product companies like Jio, Qualcomm India, and Texas Instruments Bengaluru.

C++ JOB LISTINGS IN INDIA

~82,000

Source: Naukri.com keyword search, April 2026

““OOP is not about writing less code — it is about writing code that does not break when things change.””

WATCH — VIDEO REFERENCE (ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in C++ — Week 1, Lecture 1: Motivation for Object Oriented Programming (IIT Kharagpur, Prof. Partha Pratim Chakraborty)

Link — <https://nptel.ac.in/courses/106105151>

Always link the official channel — beware fan-made or re-upload copies.

WHERE TO GO NEXT

You understand why OOP was invented. In Session 02 you will define your first real class and create objects from it.

Explore deeper — Read Chapter 1 of 'Object-Oriented Programming with C++' by E. Balaguruswamy — the most widely used OOP textbook in Indian engineering colleges.

Try on your own — Pick any app you use daily (GPay, Swiggy, IRCTC). List 3 real-world objects you can spot in it, along with their attributes and behaviours.

Connect the dots — Session 02 builds the Note class from scratch — you will define attributes, methods, and create multiple objects in main().

Going further — Research: What are the SOLID principles? How do they extend OOP best practices for large codebases?

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C++ teacher. Explain the difference between procedural programming in C and object-oriented programming in C++ using a student record system from an Indian college as the example. Keep it in simple Hinglish and under 150 words.

SESSION

02

60 MIN · CORE OOP

Classes and Objects

◆ Learning Objectives

- Define the terms 'class' and 'object' and explain the difference between them
- Write a C++ class with attributes and member functions
- Instantiate objects from a class and access their members using the dot operator
- Distinguish between a class definition and an object declaration
- Apply the Note class example to a real task management scenario

01 Class: The Blueprint

A class is a blueprint — it defines what attributes and behaviours an object will have, but by itself it creates nothing. Think of a flat plan drawn by an architect in Ahmedabad: the plan is the class; the actual flats built from it are the objects. Every flat shares the same layout (class), but each has different tenants and furniture (data).

CONCEPT — Class vs Object

Class: A user-defined data type that groups attributes (data) and methods (functions) under one name. It is the template.

Object: A specific instance of a class. Creating an object is called instantiation. Each object gets its own independent copy of all the class's attributes.

Term	What it is	Real-world analogy
Class	The blueprint / template	Flat floor plan drawn by architect
Object	A specific instance of the class	An actual flat built from that plan
Attribute	Data stored inside the object	Number of rooms, tenant name
Method	Function that belongs to the class	Open door, pay rent

02 Writing Your First Class: Note

The lecture flow uses a Note class as the running example — a note has a title, a date, and a flag tracking whether it is done. This is a perfect first class: small enough to write in five minutes, realistic enough to understand why each part matters.

```
#include <iostream>
#include <string>
using namespace std;

class Note {
public:

    string title;
    string date;
    bool    isDone;

    void display() {
cout << title << " (" << date << ") ";
cout << (isDone ? "[Done]" : "[Pending]") << endl;
    }

    void markDone() { isDone = true; }
};
```

Class declaration — defines the blueprint

Access specifier — members below are accessible from outside

Attribute: note heading

Attribute: date note was created

Attribute: task completion flag

Member function — prints a formatted summary

Member function — sets completion flag

Semicolon mandatory after class body

03 Creating Objects and Calling Methods

Once a class is defined, you create objects the same way you declare any variable — using the class name as the type. Use the dot operator (.) to access an object's attributes and to call its methods. Each object holds its own independent copy of the data.

```
int main() {
    Note n1;
    n1.title = "Finish OOP assignment";
    n1.date  = "2026-04-28";
    n1.isDone = false;

    Note n2;
    n2.title = "Submit project to TOPS";
    n2.date  = "2026-04-30";
    n2.isDone = false;

    n1.markDone();
    n1.display();
    n2.display();
    return 0;
}
```

Create object n1 of type Note

Set attribute via dot operator

Create a second, completely independent object

Call method on n1 only — n2 is unaffected

Output: Finish OOP assignment (2026-04-28) [Done]

Output: Submit project to TOPS (2026-04-30) [Pending]

TRY THIS LIVE

Type the full Note class and main() above. Add a third Note object for any personal task. Call markDone() on two of the three objects. Print all three using display(). Confirm that only the two you called markDone() on show [Done].

04 Class vs Struct: What Changed in C++?

You already know struct from C — it groups variables together. In C++, class and struct are almost identical. The only technical difference is the default access level: struct members are public by default, while class members are private by default. In practice, programmers use struct for plain data containers and class for entities that also have behaviour.

struct (C-style)

Members are public by default. Typically used for plain data grouping — no methods, no access control. Carried over from C for backwards compatibility.

class (C++ OOP)

Members are private by default. Designed for entities with both data and behaviour. Supports access modifiers, constructors, inheritance, and all OOP features.

“A class without objects is like an app with no users — the design exists, but nothing is running yet.”

WATCH — VIDEO REFERENCE (ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in C++ — Week 2: Classes, Objects and Member Functions (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105151>

WHERE TO GO NEXT

You can define a class and create objects. Next you will automate object setup using constructors.

Explore deeper — Read Chapter 3 'Classes and Objects' in Balaguruswamy's 'Object-Oriented Programming with C++'.

Try on your own — Extend your Note class: add a priority attribute (1, 2, or 3) and a method showPriority() that prints 'High', 'Medium', or 'Low' based on the value.

Connect the dots — Session 03 introduces constructors, which remove the need to set every attribute manually after creating an object.

Going further — Research: What is the 'this' pointer in C++ and when do you need to use it inside a member function?

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C++ tutor. Explain classes and objects using a real Indian example — like a Swiggy order or an HDFC bank account. Show a simple class definition and how to create two different objects from it. Use simple Hinglish. Keep it under 150 words.

SESSION

03

60 MIN · CORE OOP

Constructors & Destructors

◆ Learning Objectives

- Explain what a constructor does and why it is needed
- Write default, parameterized, and copy constructors for a class
- Describe the destructor lifecycle and when it is called
- Use constructors to initialise object data in a single line
- Apply the auto-save pattern using a destructor

01 Why Constructors Exist

In Session 02 you had to set every Note attribute manually after creating the object — three separate assignment lines per object. Constructors remove that repetition. They are special functions that run automatically the moment an object is created. Think of a new Jio SIM activation: the moment you register it, a constructor-like process sets your number, plan, and data balance without you configuring each value separately.

CONCEPT — Constructor

A constructor is a special member function with the same name as the class and no return type (not even void). It is called automatically when an object is created. Its job is to initialise the object's attributes to valid starting values so the object is ready to use immediately.

02 Types of Constructors

C++ provides three constructor types. The default constructor takes no arguments and sets safe placeholder values. The parameterized constructor accepts arguments so each object can receive its own data at creation time. The copy constructor creates a new object as an exact, independent duplicate of an existing one.

```
class Note {
public:
    string title; string date; bool isDone;

    // 1. Default Constructor
    Note() {
        title = "Untitled";
        date = "2026-01-01";
        isDone = false;
    }
}
```

Called when no arguments are given: Note n;

```
// 2. Parameterized Constructor
Note(string t, string d) {
    title = t;
    date = d;
    isDone = false;
}

// 3. Copy Constructor
Note(const Note &src) {
    title = src.title;
    date = src.date;
    isDone = src.isDone;
}
};
```

Called with arguments: Note n("Study", "2026-04-28");

Called with an existing object: Note n2(n1);

Type	How it is called	Use case
Default	Note n;	Blank note with safe placeholder values
Parameterized	Note n("OOP Study", "2026-04-28");	Note with title and date set at creation
Copy	Note n2(n1);	Independent duplicate of an existing note

```
int main() {
    Note n1;
    Note n2("OOP Session 3", "2026-04-28");
    Note n3(n2);
    cout << n1.title << endl;
    cout << n2.title << endl;
    cout << n3.title << endl;
    n2.title = "Changed";
    cout << n3.title << endl;
}
```

Default constructor — title = 'Untitled'

Parameterized constructor

Copy constructor — n3 is an independent clone of n2

Output: Untitled

Output: OOP Session 3

Output: OOP Session 3

Only n2 changes — n3 is independent

Output: OOP Session 3 (unchanged)

TRY THIS LIVE

Add all three constructors to your Note class. In main(), create one object with each constructor type. Print the title of each. Then change n2.title to something new and print both n2 and n3 — confirm that n3 was not affected. A copy constructor creates an independent copy, not a reference.

03 The Destructor

A destructor is the mirror of a constructor — it runs automatically when an object is destroyed (goes out of scope or is explicitly deleted). Its job is to release resources: closing a file, freeing memory, or saving final state. The destructor has the same name as the class but with a tilde (~) prefix, and takes no arguments.

CONCEPT — Destructor

A destructor is called automatically at the end of an object's lifetime. It is useful for cleanup — closing file handles, saving session data, or releasing heap memory. Each class can have only one destructor (no overloading). If you do not write one, C++ provides a default empty destructor.

```
class Note {
public:
    string title;
    Note(string t) { title = t; }           Parameterized constructor

    ~Note() {                               Destructor — same name as class, prefixed with ~
        cout << "Auto-saving '" << title << "' to disk..." << endl;
        Simulates save-on-exit
    }
};

int main() {
    Note n("Daily Goals");                  Constructor called — object created
    // ... work with n ...
} // <-- Destructor fires here automatically  Output: Auto-saving 'Daily Goals' to disk...
```

04 Auto-Save on Exit

The lecture flow example — auto-save user data upon exit — maps perfectly onto the destructor. When a Note object goes out of scope at the end of main(), the destructor fires and writes data to a file. The user never has to remember to call a save() function — the object manages its own cleanup. This pattern is the foundation of the Creator Dashboard mini project.

PRO TIP — Rule of Three

If your class needs a custom destructor (because it manages a resource), you almost always also need a custom copy constructor and a custom assignment operator. This is the 'Rule of Three' in C++ and prevents subtle data corruption bugs when objects are copied or assigned.

“A constructor sets the stage; a destructor cleans it up — together they make every object a self-managing unit.”

WATCH — VIDEO REFERENCE (ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in C++ — Week 3: Constructors and Destructors (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105151>

WHERE TO GO NEXT

You can now build self-initialising, self-cleaning objects. Next you will learn how one class can inherit the work of another.

Explore deeper — Read Chapter 5 'Constructors and Destructors' in Balaguruswamy's OOP with C++.

Try on your own — Add a parameterized constructor to Note that also accepts a bool for isDone. Create a note in a single line: `Note n("Revise Pointers", "2026-04-29", false);`

Connect the dots — Session 04 introduces Inheritance — where a derived class can call a base class constructor to set up shared data automatically.

Going further — Research: What is a 'delegating constructor' introduced in C++11 and how does it reduce code duplication between constructor overloads?

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C++ tutor. Explain constructors and destructors using an HDFC bank account as the example. Cover default constructor, parameterized constructor, and destructor with a short code example. Use simple Hinglish. Keep it under 150 words.

SESSION

04

60 MIN · CORE OOP

Inheritance

◆ Learning Objectives

- Explain what inheritance is and why it reduces code duplication
- Implement single, multilevel, and hierarchical inheritance in C++
- Use public inheritance access specifier correctly
- Write the ContentCreator -> YouTuber / Podcaster hierarchy from the lecture flow
- Identify when to use inheritance versus creating separate unrelated classes

01 What is Inheritance?

Inheritance lets a new class (the derived class) automatically receive all the attributes and methods of an existing class (the base class), then add or override behaviour on top. Think of how Jio and Airtel are both telecom providers — they share common features (calls, SMS, data) but each adds its own services. In code, a Jio class can inherit from a TelecomProvider base class instead of rewriting shared logic from scratch.

CONCEPT — Inheritance

Inheritance is the OOP mechanism where a derived class acquires the properties and behaviours of a base class. Syntax: `class Derived : access_specifier Base { ... }`. It models an IS-A relationship — a YouTuber IS-A ContentCreator. The derived class can use all public and protected members of the base class as if they were its own.

02 Types of Inheritance in C++

Type	Structure	Example from lecture flow
Single	One base, one derived	ContentCreator -> YouTuber
Multilevel	A inherits B, which inherits C	Person -> ContentCreator -> YouTuber
Hierarchical	One base, multiple derived	ContentCreator -> YouTuber AND Podcaster
Multiple	Derived inherits from two base classes	SocialCreator inherits User + ContentCreator
Hybrid	Combination of above types	Complex real-world enterprise hierarchies

03 Writing the ContentCreator Hierarchy

The lecture flow uses a content creator hierarchy as its running example: a ContentCreator base class, with YouTuber and Podcaster as derived classes. Both share attributes like name and followerCount, but each adds its own platform-specific attributes and methods.

```
class ContentCreator {
    public:
        string name;
        int    followerCount;
        void introduce() {
            cout << "Hi, I am " << name << " with " << followerCount << " followers." << endl;
        }
};
```

Base class — shared by all creator types

```
class YouTuber : public ContentCreator {
    public:
        int videoCount;
        void uploadVideo(string title) {
            cout << name << " uploaded: " << title << endl;
            name is inherited — no need to redeclare it
        }
};
```

Derived class — inherits everything from ContentCreator

YouTuber-specific attribute

YouTuber-specific method

```
class Podcaster : public ContentCreator {
    public:
        string podcastName;
        void releaseEpisode(string ep) {
            cout << name << " released episode: " << ep << endl;
        }
};
```

Another derived class

```
int main() {
    YouTuber yt;
    yt.name          = "TechWithNiral";
    yt.followerCount = 50000;
    yt.videoCount    = 120;
    yt.introduce();
    yt.uploadVideo("OOP in 10 Minutes");
}
```

Inherited attribute from ContentCreator

Inherited attribute

YouTuber's own attribute

Calls inherited method — Output: Hi, I am TechWithNiral...

Calls YouTuber's own method

TRY THIS LIVE

Type the full hierarchy above. Add a Podcaster object in `main()` with your own data. Call `introduce()` on it — Podcaster does not define `introduce()`, but it inherits it from `ContentCreator` and works perfectly. Then add a third derived class: `Blogger`, with attributes `website (string)` and `postCount (int)`.

04 Access Specifiers in Inheritance

When you write `class YouTuber : public ContentCreator`, the access specifier controls how the base class's members appear in the derived class. Public inheritance — by far the most common — keeps the original access levels intact. Private inheritance is occasionally used when you want to reuse internal code without exposing the base class interface to users of the derived class.

Base member	public inheritance	protected inheritance	private inheritance
public	public in derived	protected in derived	private in derived
protected	protected in derived	protected in derived	private in derived
private	Not accessible	Not accessible	Not accessible

CASE STUDY — Zoho CRM Entity Hierarchy

Context — Zoho Corporation, headquartered in Chennai, builds one of India's largest SaaS portfolios. Zoho CRM is used by over 250,000 businesses worldwide to manage Leads, Contacts, Accounts, and Deals.

Challenge — Each CRM entity type shares common fields — name, phone, email, owner — but has very different workflows, validation rules, and data models. Maintaining duplicate code for each entity was becoming unsustainable as the product scaled.

What they did — Zoho's engineering team structured their data model using inheritance: a base `CRMEntity` class holds shared fields and common methods like `validate()` and `syncToServer()`. `Lead`, `Contact`, `Account`, and `Deal` are derived classes that add their own specialised attributes and override lifecycle methods.

Result — New entity types could be added without touching the shared logic. Bug fixes in the base class automatically applied to all derived entities, significantly reducing maintenance overhead across the product.

Takeaway — Inheritance is not just a textbook concept — it is how real Indian software products manage complexity as they scale to millions of users.

Source: *Zoho Engineering Blog*, zoho.com

“Inheritance is the art of saying: everything my parent knows, I know too — plus a few things they don't.”

WATCH — VIDEO REFERENCE (ENGLISH)**Channel** — NPTEL (official — IITs / IISc)**Watch** — Programming in C++ — Week 5: Inheritance and Derived Classes (IIT Kharagpur)**Link** — <https://nptel.ac.in/courses/106105151>**WHERE TO GO NEXT**

You can now build class hierarchies that share and extend behaviour. Next, you will make derived classes respond differently to the same method call — that is Polymorphism.

Explore deeper — Read Chapter 8 'Inheritance' in Balaguruswamy's OOP with C++.

Try on your own — Build the multilevel version: create a Person base class, ContentCreator as a class that inherits Person, and YouTuber as a class that inherits ContentCreator. Test that a YouTuber object can access attributes from all three levels.

Connect the dots — Session 05 builds directly on the ContentCreator hierarchy — the uploadContent() method will behave differently for YouTuber vs Podcaster using polymorphism.

Going further — Research: What is the 'Diamond Problem' in C++ multiple inheritance and how does the 'virtual' keyword in the inheritance declaration solve it?

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C++ teacher. Explain inheritance using an Indian example — like different types of Reliance Group employees or Zomato delivery agents vs restaurant partners. Show single inheritance and hierarchical inheritance with short code. Use simple Hinglish. Keep it under 150 words.

SESSION

05

60 MIN · CORE OOP

Polymorphism

◆ Learning Objectives

- Define polymorphism and explain its two main types
- Implement function overloading for compile-time polymorphism
- Use virtual functions and function overriding for runtime polymorphism
- Explain why the 'virtual' keyword is necessary for dynamic dispatch
- Apply polymorphism to the uploadContent() example from the lecture flow

01 What is Polymorphism?

Polymorphism means 'many forms'. In programming, the same function name can behave differently depending on how it is called or which object calls it. Think of a UPI payment: whether you pay through PhonePe, GPay, or Paytm, you perform the same action — pay() — but each app executes it differently under the hood. That is polymorphism in real life.

CONCEPT — Two Types of Polymorphism

Compile-Time (Static): The compiler decides which function to call before the program runs, based on argument types or count. Achieved using function overloading.

Runtime (Dynamic): The decision is made while the program is running, based on the actual type of the object. Achieved using virtual functions and function overriding. Also called dynamic dispatch or late binding.

02 Compile-Time: Function Overloading

Function overloading lets you define multiple functions with the same name but different parameter lists. The compiler picks the correct version based on the arguments you provide at the call site. This is resolved entirely at compile time — no extra cost at runtime.

```
class ContentCreator {
public:
    string name;

    // Three overloaded versions of uploadContent
    void uploadContent(string title) {           Version 1 — title only
cout << name << " uploaded: " << title << endl;
    }
}
```

```
void uploadContent(string title, int durationMins) {
    Version 2 — title + duration
    cout << name << " uploaded '" << title << "' (" << durationMins << " mins)" << endl;
}
void uploadContent(string title, string platform) {
    Version 3 — title + platform
    cout << name << " posted '" << title << "' on " << platform << endl;
}
};

int main() {
    ContentCreator c;  c.name = "Niral";
    c.uploadContent("OOP Basics");           Calls version 1
    c.uploadContent("OOP Basics", 15);       Calls version 2 — int argument
    c.uploadContent("OOP Basics", "Instagram");
                                           Calls version 3 — string argument
}
```

03 Runtime: Function Overriding and Virtual Functions

Runtime polymorphism goes further: a base class pointer can point to a derived class object, and when you call a virtual function through that pointer, C++ figures out at runtime which version to call. Without the virtual keyword, C++ always calls the base class version regardless of which object the pointer actually holds.

```
class ContentCreator {                                Base class
public:
    string name;
    virtual void uploadContent() {                    'virtual' enables runtime polymorphism
    cout << name << " uploads generic content." << endl;
    }
};

class YouTuber : public ContentCreator {
public:
    void uploadContent() override {                  'override' confirms we are replacing the base version
    cout << name << " uploads a YouTube video." << endl;
    }
};

class Podcaster : public ContentCreator {
public:
    void uploadContent() override {
    cout << name << " publishes a podcast episode." << endl;
}
```

```

    }
};

int main() {
    ContentCreator* p;           Base class pointer
    YouTuber yt;   yt.name = "Niral";
    Podcaster pod; pod.name = "Dhruv";
    p = &yt;   p->uploadContent();   Output: Niral uploads a YouTube video.
    p = &pod;   p->uploadContent();   Output: Dhruv publishes a podcast episode.
}

```

CONCEPT — Why 'virtual' Matters

Without virtual, calling uploadContent() through a base class pointer always runs the base class version — even when the pointer points to a YouTuber or Podcaster. With virtual, C++ uses a vtable (virtual function table) to perform dynamic dispatch at runtime, calling the correct version for the actual object type.

TRY THIS LIVE

Copy the runtime polymorphism code above. Remove the 'virtual' keyword from ContentCreator::uploadContent(). Run it. Both calls now print the generic base class message — even for YouTuber and Podcaster. Add 'virtual' back and run again. This single test makes the purpose of virtual unmistakably clear.

Compile-Time Polymorphism

Resolved at compile time. Uses function overloading or operator overloading. No runtime overhead. Does not need virtual keyword. Also called static binding or early binding.

Runtime Polymorphism

Resolved during program execution. Uses virtual functions and overriding via base class pointer or reference. Enables plugin-style design. Also called dynamic binding or late binding.

“Polymorphism is what lets you write code that works correctly with objects you have not designed yet.”

WATCH — VIDEO REFERENCE (ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in C++ — Week 7: Polymorphism and Virtual Functions (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105151>

WHERE TO GO NEXT

You can now write classes that respond intelligently to the same function call. Next you will learn to protect internal data with encapsulation.

Explore deeper — Read Chapter 9 'Polymorphism' in Balaguruswamy's OOP with C++.

Try on your own — Add a third derived class — `InstagramCreator` — with its own `uploadContent()` that prints a reel-specific message. Store all three objects in a `ContentCreator*` array and loop through calling `uploadContent()` on each. Watch polymorphism work in sequence.

Connect the dots — Session 06 — Encapsulation and Abstraction — will protect the data inside your content creator hierarchy using private members and pure virtual functions.

Going further — Research: What is a 'pure virtual function'? How does it force every derived class to provide its own implementation, and what does it turn the base class into?

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C++ tutor. Explain polymorphism — both compile-time (function overloading) and runtime (virtual functions) — using Indian apps like Swiggy, Zomato, and Blinkit as examples. Show short code. Use simple Hinglish. Keep it under 150 words.

SESSION

06

60 MIN · CORE OOP

Encapsulation & Abstraction

◆ Learning Objectives

- Apply public, private, and protected access modifiers correctly
- Implement encapsulation using private attributes and public getter/setter methods
- Validate data inside setter methods to prevent invalid object state
- Explain the difference between encapsulation and abstraction
- Write an abstract class with a pure virtual function using the Platform example

01 Encapsulation: Data Hiding

Encapsulation bundles an object's data with the methods that operate on it, and restricts direct access to that data from outside the class. Think of your UPI PIN: the GPay app bundles your PIN with a `verifyPIN()` function — but the PIN itself is never directly accessible from outside the app's internal class. That protection is encapsulation.

CONCEPT — Encapsulation

Encapsulation hides the internal state of an object. Attributes are declared private — not directly readable or writable from outside the class. Public methods called getters and setters provide controlled, validated access. This prevents accidental or malicious corruption of an object's state.

```
class ContentCreator {
    private:
        string name;
        int    followerCount;

    public:
        string getName()    { return name; }
        int    getFollowers(){ return followerCount; }

        void setName(string n) {
            if (n.length() > 0) name = n;
            else cout << "Error: Name cannot be empty!" << endl;
        }
        void setFollowers(int f) {
            if (f >= 0) followerCount = f;
            else cout << "Error: Count must be positive!" << endl;
        }
};
```

These attributes CANNOT be accessed directly from outside

Protected from direct modification

These methods ARE the controlled access points

Getter — read-only access

Setter with validation

Only accept non-empty names

Follower count cannot be negative

Access Modifier	Visible within class?	Visible in derived class?	Visible outside class?
public	Yes	Yes	Yes
protected	Yes	Yes	No
private	Yes	No	No

TRY THIS LIVE

Make name and followerCount private in your ContentCreator class. Try to access them directly in main() with creator.name = "Test" — the compiler will refuse with an error. Then use creator.setName("Test") and confirm it works. Try passing an empty string to setName() and watch the validation message fire.

02 Abstraction: Hiding Complexity

Abstraction means showing only what is necessary and hiding how things work internally. When you tap 'Recharge' on Paytm, you don't see the payment gateway handshake, bank API calls, or telecom provider integration happening behind the scenes. The interface is simple; the complexity is hidden. In code, you define a clean method signature and hide the messy implementation details from the user of your class.

CONCEPT — Abstraction vs Encapsulation

Encapsulation = HOW you hide data. You use private/public keywords and getter/setter methods to control access to an object's state.

Abstraction = WHAT you hide. You define a clean interface (what the user needs to know) and conceal the implementation (how it works internally). Abstract classes enforce this by specifying a required method signature without providing the body.

03 Abstract Classes and Pure Virtual Functions

An abstract class cannot be instantiated directly — it exists only to be inherited. It contains at least one pure virtual function, declared with = 0 at the end. Every derived class must provide its own implementation of that function. This is how C++ enforces a contract: every class that extends Platform must implement upload().

```
class Platform {
    public:
        string platformName;
```

Abstract base class — cannot create Platform objects directly

```

    virtual void upload(string content) = 0;

    void showPlatform() {
    cout << "Platform: " << platformName << endl;
    }
    virtual ~Platform() {}
};

class YouTube : public Platform {
    public:
        void upload(string content) override {
    cout << "YouTube: Uploading video '" << content << "'" << endl;
        }
};

class Instagram : public Platform {
    public:
        void upload(string content) override {
    cout << "Instagram: Posting reel '" << content << "'" << endl;
        }
};

// Platform p;  <-- ERROR: cannot instantiate abstract class
YouTube yt;  yt.platformName = "YouTube";
yt.upload("OOP Tutorial Part 1");

```

Pure virtual — = 0 makes this class abstract

Concrete method — shared by all derived classes

Virtual destructor — best practice for abstract classes

MUST implement the pure virtual function

Derived class CAN be instantiated

Output: YouTube: Uploading video 'OOP Tutorial Part 1'

Encapsulation

Protects data using private modifiers and getter/setter methods. Prevents invalid state from being set. The object is still fully usable by anyone who calls its public methods correctly.

Abstraction

Hides implementation complexity behind a clean interface. Defines what a class must do via abstract methods, not how it does it. Forces derived classes to implement a standard interface.

“Encapsulation locks the safe; abstraction hides the fact that a safe even exists.”

WATCH — VIDEO REFERENCE (ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in C++ — Week 6: Encapsulation, Access Specifiers and Abstract Classes (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105151>

WHERE TO GO NEXT

You have now covered all four OOP pillars. Session 07 applies them to a real problem: persisting object data to files.

Explore deeper — Read Chapter 6 'Data Hiding and Encapsulation' in Balaguruswamy's OOP with C++.

Try on your own — Add a private attribute earnings (float) to ContentCreator. Write a getter and a setter that validates the value is positive. Test it with both valid (50000.0) and invalid (-100.0) inputs.

Connect the dots — Session 07 (File Handling) will use the ContentCreator class from this session to save and load creator analytics data to and from a text file.

Going further — Research: What is an interface in Java or C#? How is it similar to and different from an abstract class in C++?

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C++ tutor. Explain the difference between encapsulation and abstraction using an Indian banking app (like HDFC or SBI mobile app). Show a simple ContentCreator class with private attributes and public getters/setters. Use simple Hinglish. Keep it under 150 words.

SESSION

07

60 MIN · APPLIED OOP

File Handling in C++

◆ Learning Objectives

- Explain why file handling is essential for data persistence
- Use ofstream to write data to a text file
- Use ifstream to read data back from a text file line by line
- Apply file open modes (ios::app, ios::trunc) appropriately
- Integrate file handling into the ContentCreator class to save and load analytics

01 Why File Handling?

Every program written so far loses all its data when it exits — objects and variables live only in RAM. File handling solves this by writing data to permanent storage on disk. Think of the Creator Dashboard you will build in the mini project: without file handling, every creator and content item disappears the moment the user closes the console. With file handling, data persists and the next run picks up exactly where the last one left off.

CONCEPT — File Streams

C++ treats files as streams — just like cin and cout are streams for keyboard and screen. Three stream classes from the <fstream> header:

ofstream — output file stream. Writes data TO a file.

ifstream — input file stream. Reads data FROM a file.

fstream — combined stream for both reading and writing.

02 Writing to a File with ofstream

Opening a file for writing is as simple as declaring an ofstream variable and calling open() with the filename. You then use the << operator — the same one you use with cout — to send data to the file. Always check whether the file opened successfully, and always call close() when done to flush the buffer.

```
#include <fstream>
#include <iostream>
using namespace std;
```

Required header for file streams

```
int main() {
    ofstream outFile;
    outFile.open("analytics.txt");
```

Declare an output file stream

Creates the file; overwrites if it already exists

```

    if (!outFile) {
        cout << "Error: Could not open file!" << endl;
        return 1;
    }

    outFile << "Creator: TechWithNiral" << endl;
    Write a line — same syntax as cout
    outFile << "Platform: YouTube" << endl;
    outFile << "Views: 45000" << endl;
    outFile.close();
    cout << "Saved to analytics.txt" << endl;
    return 0;
}

```

Always check — file may be locked or path invalid

Exit with error code

Always close — flushes the write buffer

03 Reading from a File with ifstream

```

int main() {
    ifstream inFile;
    inFile.open("analytics.txt");

    if (!inFile) {
        cout << "File not found!" << endl;
        return 1;
    }

    string line;
    while (getline(inFile, line)) {
        cout << line << endl;
    }
    inFile.close();
    return 0;
}

```

Declare an input file stream

Open the file created in the previous program

Read one line at a time until end of file

Print each line to the console

Mode	Flag	Behaviour
Default write	ios::out (default for ofstream)	Creates file or overwrites all existing content
Append	ios::app	Adds new content at end — preserves existing data

Mode	Flag	Behaviour
Read	ios::in (default for ifstream)	Opens file for reading only; fails if file missing
Binary	ios::binary	Read/write raw bytes — used for non-text (image/exe) files
Truncate	ios::trunc	Explicitly deletes all existing content before writing

TRY THIS LIVE

Run the write program, then run the read program and verify the output. Open analytics.txt in VS Code to confirm the content. Then change `outFile.open("analytics.txt")` to `outFile.open("analytics.txt", ios::app)` and run the write program again — the new lines should be added after the existing ones, not replace them.

04 Saving Creator Analytics with OOP

Now we combine OOP and file handling. The `ContentCreator` class gets two new methods: `saveToFile()` writes the object's encapsulated data to a CSV file, and we can write a standalone `loadAll()` function that reads the file and reconstructs objects. This pattern is the direct foundation of the Creator Dashboard mini project.

```
class ContentCreator {
private:
string name; string platform; int views;
public:
ContentCreator(string n, string p, int v) {
    name = n; platform = p; views = v;
}

void saveToFile(string filename) {
    ofstream f(filename, ios::app);           Append mode — don't overwrite other creators
    if (f) f << name << "," << platform << "," << views << endl;
    CSV format
    f.close();
}

void display() const {
    cout << name << " | " << platform << " | Views: " << views << endl;
}
};

int main() {
```

```
ContentCreator c1("TechWithNiral", "YouTube", 45000);
ContentCreator c2("FinanceGuru", "Instagram", 82000);
    c1.saveToFile("creators.csv");           Writes c1 to file
    c2.saveToFile("creators.csv");           Appends c2 after c1
    c1.display(); c2.display();
}
```

CASE STUDY — Log-Based Analytics at Jio Platforms

Context — Jio Platforms, headquartered in Mumbai, serves over 450 million subscribers and generates one of the largest data streams in India — tracking app events, network performance, and billing data in real time.

Challenge — Jio's engineering teams needed a lightweight, persistent way to log analytics events from hundreds of microservices without introducing database overhead for every individual write operation.

What they did — Engineers implemented a file-stream-based logging layer where each service writes structured event records to append-mode log files at high speed. Downstream batch processors later read these files for analytics aggregation and billing calculations.

Result — Write latency dropped significantly compared to synchronous database inserts, and the file-based buffer acted as a reliable intermediate layer between real-time events and structured database storage.

Takeaway — The ofstream append pattern you learned today is the same core mechanism behind industrial-grade logging systems — just scaled up by many orders of magnitude.

Source: Jio Platforms, jio.com

“RAM is a whiteboard; a file is a notebook. Good programs know the difference.”

WATCH — VIDEO REFERENCE (ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in C++ — Week 9: File Handling with Streams (IIT Kharagpur)

Link — <https://nptel.ac.in/courses/106105151>

WHERE TO GO NEXT

You now have all the tools to build a complete OOP application with persistent storage. The mini project brings everything together in one program.

Explore deeper — Read Chapter 12 'File Handling' in Balaguruswamy's OOP with C++.

Try on your own — Add a loadFromFile() function that reads creators.csv line by line, splits each line by comma, and prints the parsed data in a formatted table using setw() for alignment.

Connect the dots — Mini Project 2 — Creator Dashboard Lite — uses exactly this file-handling pattern to persist content items across sessions, combined with all four OOP pillars from the module.

Going further — Research: What is binary file handling in C++ (`ios::binary`)? When is it faster or safer than text-mode file handling for storing complex objects?

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a C++ tutor. Explain file handling using `ofstream` and `ifstream` with a practical example of saving and reading a student's daily study log. Show short code. Use simple Hinglish. Keep it under 150 words.

REVIEW



INTERVIEW PREP · MODULE 4

Interview Questions

◆ Introduction to OOP Programming (C++)

Rehearse each answer out loud, aiming for about 60 seconds. For code questions, walk through your reasoning — interviewers value clear thinking as much as correct syntax.

1

What is the main difference between procedural programming and object-oriented programming (OOP)?

Think about how data and functions are treated in each approach.

MODEL ANSWER

In procedural programming (like C), code is organized as a sequence of functions that operate on global or shared data — any function can read or modify that data. In OOP, data and the functions that operate on it are bundled together inside objects, which hides internal details and controls access. OOP makes large programs easier to manage, extend, and maintain because related data and behaviour are kept together. C++ is an example of a language that supports both styles.

2

Why is global data in procedural languages like C considered a limitation?

Consider what happens to data security and integrity when any part of the program can access or modify it.

MODEL ANSWER

Global data can be read or modified by any function in the program, which makes it hard to trace bugs — a variable may be corrupted by a function that has nothing to do with it logically. As a program grows, this creates tight coupling where changing one part of the code can unexpectedly break another. OOP solves this with encapsulation, restricting access to data through controlled class interfaces. This makes the code safer, more modular, and easier to maintain.

3

What is a class in OOP, and how does it differ from an object?

One is a blueprint, the other is a real instance.

MODEL ANSWER

A class is a blueprint or template that defines the properties (data members) and behaviours (member functions) that objects of that type will have. An object is a specific instance of a class — it occupies memory and holds actual values. For example, a `Car` class might define `brand` and `speed`, while `myCar` is an object with `brand = 'Honda'` and `speed = 120`. You can create many objects from one class, just as a factory can produce many cars from one design.

4

What is a constructor in OOP, and when is it called?

Think about what happens when you create a new object.

MODEL ANSWER

A constructor is a special member function with the same name as the class and no return type. It is called automatically when an object is created, making it the ideal place to initialize data members to valid starting values. Without a constructor, data members may contain garbage values. C++ provides a default constructor (no parameters) automatically if you don't define one, but you can write parameterized constructors to accept initial values at creation time.

5

Explain the difference between a default constructor and a parameterized constructor.

MODEL ANSWER

A default constructor takes no arguments and initializes data members to default values — it is called when you write `Note n1;`. A parameterized constructor accepts arguments so you can initialize the object with specific values at creation — `Note n1("Shopping List", "2025-01-01");`. Having parameterized constructors makes objects immediately valid and useful without requiring separate setter calls after creation. A class can have both types through constructor overloading.

6

Write a parameterized constructor for a class `Content` that sets the title and platform.

MODEL ANSWER

The class definition would be: `class Content { public: string title; string platform; Content(string t, string p) { title = t; platform = p; } };`. When you create an object: `Content c1("Vlog 1", "YouTube");`, the constructor runs immediately and sets `title` to `"Vlog 1"` and `platform` to `"YouTube"`. Using `this->title = t;` is equivalent and makes the code clearer when parameter names match member names.

7

When is a destructor called in the lifecycle of an object?

Consider what happens when an object goes out of scope or is deleted.

MODEL ANSWER

A destructor is called automatically when an object goes out of scope (at the end of a function or block) or when `delete` is used on a dynamically allocated object. It has the same name as the class prefixed with a tilde: `~ClassName()`. Destructors are used to release resources the object holds — closing files, freeing dynamically allocated memory, or saving data before the object is destroyed. Unlike constructors, there can only be one destructor and it takes no arguments.

8

What is inheritance in OOP, and how does it help with code reuse?

MODEL ANSWER

Inheritance allows a new class (derived/child class) to acquire the properties and methods of an existing class (base/parent class), reusing already-written code. For example, if `ContentCreator` has common attributes like `name` and `followersCount`, a `Youtuber` class can inherit from it and add YouTube-specific features like `channelName`. This avoids writing the same code twice and makes the hierarchy of related classes easy to understand. The child class can also override inherited methods to change their behaviour.

9

Explain the difference between single inheritance and multilevel inheritance with examples.

MODEL ANSWER

Single inheritance means one class inherits from exactly one base class — for example, ``YouTuber`` inherits from ``ContentCreator``. Multilevel inheritance is a chain: ``ContentCreator`` is the base, ``YouTuber`` inherits from it, and ``ShortFormYouTuber`` inherits from ``YouTuber``. Each level extends the one above. In contrast, hierarchical inheritance means multiple classes inherit from the same base — both ``YouTuber`` and ``Podcaster`` inherit from ``ContentCreator``. Knowing which type to use depends on the real-world relationship between the entities.

10

What is polymorphism in OOP?

Think about how the same function name can behave differently.

MODEL ANSWER

Polymorphism means 'many forms' — the same function name can behave differently depending on context. Compile-time (static) polymorphism is achieved through function overloading, where multiple functions share a name but differ in their parameter types or count. Runtime (dynamic) polymorphism is achieved through function overriding, where a derived class redefines a base class method, and the correct version is selected at runtime based on the actual object type. Polymorphism enables flexible, extensible designs where new classes can be added without changing existing code.

11

Suppose you have a method `uploadContent()` that behaves differently for YouTube and Instagram. How would you achieve this using polymorphism?

MODEL ANSWER

Declare `uploadContent()` as a virtual function in the base ``Platform`` class. Both ``YouTube`` and ``Instagram`` derive from ``Platform`` and override `uploadContent()` with their own implementation — YouTube might set resolution to 1080p, while Instagram resizes to a square format. At runtime, when you call `uploadContent()` through a ``Platform*`` pointer, C++ automatically calls the correct version based on the actual object type. This is runtime polymorphism via virtual functions, also known as dynamic dispatch.

12

What are access modifiers in OOP, and why are they important?

Consider how you control which parts of your code can access certain data.

MODEL ANSWER

Access modifiers control the visibility of class members from outside the class. ``public`` members are accessible from anywhere. ``private`` members are accessible only within the class itself — they cannot be read or changed directly from outside. ``protected`` members are accessible within the class and its derived classes. They are important because they enforce encapsulation — by making data members ``private`` and providing controlled ``public`` methods (getters/setters), you protect data integrity and hide implementation details.

13

What is encapsulation, and how do access modifiers help achieve it?**MODEL ANSWER**

Encapsulation is the OOP principle of bundling data and the methods that operate on it inside a class, while hiding internal details from the outside world. Access modifiers enforce this — by declaring data members as `private`, you prevent external code from directly changing them. Instead, controlled public methods (like `setTitle()` and `getTitle()`) provide a safe interface. This means you can change the internal implementation without affecting the rest of the program, and you can add validation inside setter methods to prevent invalid data.

14

Define abstraction in OOP and explain the purpose of abstract classes.*Focus on hiding implementation details and showing only the necessary features.***MODEL ANSWER**

Abstraction means exposing only what is necessary and hiding internal complexity. For example, you use `printf()` without knowing how it formats and outputs text internally. In C++, an abstract class contains at least one pure virtual function — a function declared with `= 0` that has no body. Abstract classes cannot be instantiated directly; they serve as contracts that force all derived classes to provide their own implementation. This guarantees a consistent interface while allowing each subclass to implement its own behaviour.

15

What is a pure virtual function? Give an example.**MODEL ANSWER**

A pure virtual function is a function in a base class declared with `= 0`, indicating it has no implementation in the base class and must be overridden in any concrete derived class. For example: `virtual void upload() = 0;` inside a `Platform` class. Any class that inherits from `Platform` must define its own `upload()` method, or it also becomes abstract. This design forces derived classes to implement the required behaviour, ensuring a uniform interface across all subclasses.

16

What is file handling in C++ and which stream classes are used for reading and writing files?*Think about how programs store data even after they close.***MODEL ANSWER**

File handling allows a C++ program to read from and write to files on disk, so data persists after the program closes. The three main stream classes are `ifstream` (input file stream) for reading from files, `ofstream` (output file stream) for writing to files, and `fstream` for both reading and writing. You open a file with `.open("filename")`, perform read/write operations, and always close with `.close()`. File handling is essential for any application that needs to save user data between sessions.

17

Imagine you are building 'Creator Dashboard Lite'. What classes and features would you include to manage content items?

Think about what data a content item needs and what actions a user might perform.

MODEL ANSWER

The core class would be `Content` with members: `title`, `platform`, `views`, and `status` (draft/published). A `Dashboard` class would manage a collection of `Content` objects and expose methods to add, edit, delete, and display items. A menu-driven `main()` function would present options and call the appropriate Dashboard method. File handling (using `ofstream`/`ifstream`) would save and load the content list. Using inheritance, you could create specialized classes like `YoutubeContent` and `InstagramContent` that extend `Content` with platform-specific fields.

18

Describe how you would implement the ability to add, edit, and delete content items in your Creator Dashboard Lite project.

MODEL ANSWER

For adding: prompt the user for title, platform, views, and status, create a `Content` object, and push it into a `vector<Content>`. For editing: display the list with indices, ask the user to choose one, then prompt for new values and update the object at that index. For deleting: show the list, ask for an index, and use `vector.erase()` to remove the item. After each operation, write the entire updated vector to the file so changes are saved immediately.

19

Integrative: How does encapsulation support file handling when saving and loading private data members of a class?

MODEL ANSWER

Since data members are `private`, external code cannot directly access them to write to a file. Instead, you provide a `public` method like `saveToFile(ofstream &out)` that writes the private members itself. Similarly, a `loadFromFile(ifstream &in)` method reads values from the file and sets the private members using the class's own logic. This keeps the internal data structure protected even during persistence operations, and if you change how data is stored internally, only those methods need updating — not every place that writes to the file.

20

Integrative: In your Creator Dashboard, how would you use inheritance and polymorphism to support multiple content platforms with different upload behaviours?

MODEL ANSWER

Create an abstract base class `Platform` with a pure virtual function `void uploadContent() = 0`. Derive `YouTube`, `Instagram`, and `Podcast` from it, each implementing `uploadContent()` differently — YouTube validates video resolution, Instagram resizes to square, Podcast sets audio bitrate. Store content items as `Platform*` pointers in a vector. When iterating and calling `uploadContent()`, C++ dispatches the correct implementation at runtime. This design lets you add a new platform like `LinkedIn` without changing any existing code — just add a new derived class.